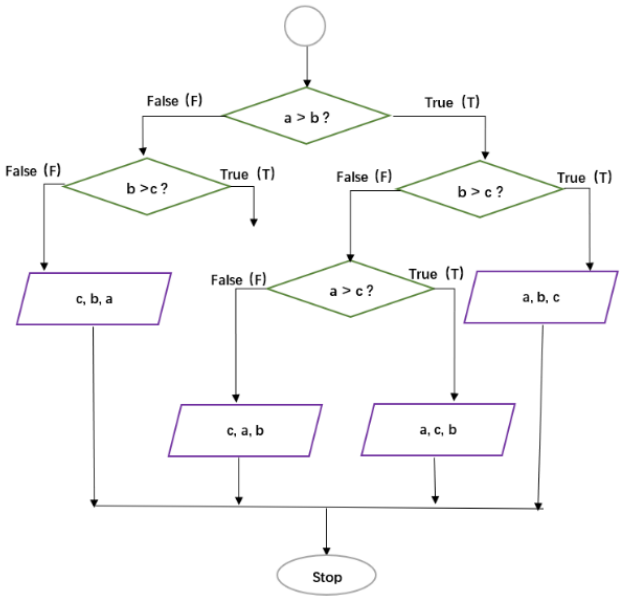
作业 1

1. Flowchart

[10 points] Write a function `Print_values` with arguments `a`, `b`, and `c` to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random `a`, `b`, and `c` values.




```
def Print_values(a, b, c):
    if a > b:
        if b > c:
            print (a, b, c)
        else:
            if a > c:
                print (a, c, b)
            else:
                print (c, a, b)
    else:
        if b > c:
            return
        else:
            print (c, b, a)
```

本题运用了函数定义、嵌套“if-else”判断、“return”结束和“print”输出。

作业 2

2. Matrix multiplication

2.1 [5 points] Make two matrices `M1` (`5` rows and `10` columns) and `M2` (`10` rows and `5` columns); both are filled with random integers from `0` and `50` .

2.2 [10 points] Write a function `Matrix_multip` to do matrix multiplication, *i.e.*, `M1 * M2` . Here you are **ONLY** allowed to use `for` loop, `*` operator, and `+` operator.

```
import random

#生成 M1:5 行 10 列，元素为 0~50 的随机整数
M1 = []
for i in range(5):
    row = []
    for j in range(10):
        row.append(random.randint(0, 50))
    M1.append(row)

#生成 M2:10 行 5 列，元素为 0~50 的随机整数
M2 = []
for i in range(10):
    row = []
    for j in range(5):
        row.append(random.randint(0, 50))
    M2.append(row)


def Matrix_multip(M1, M2):
    #创建一个五行五列的全 0 矩阵
    C = []
    for i in range(5):
        row = []
        for j in range(5):
            row.append(0)
        C.append(row)

    #for 循环计算矩阵乘法
    for i in range(5):
        for j in range(5):
            total = 0
            for k in range(10):
                total = total + M1[i][k] * M2[k][j]
            C[i][j] = total
```

```
    return C

#打印结果
result = Matrix_multip(M1, M2)
print(result, "= M1 * M2")
```

本题运用了随机数生成（“random.randint”)、列表（“for”+“append”)、嵌套“for”循环、函数定义、列表索引和“return”语句。通过三层“for”循环，使用乘法“*”和加“+”完成矩阵乘法，并输出计算结果。

作业 3

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function `Pascal_triangle` with an argument `k` to print the kth line of the Pascal triangle. Report `Pascal_triangle(100)` and `Pascal_triangle(200)`.

```
import math
def Pascal_triangle(k):
    row = []
    for i in range(k + 1):
        x = math.factorial(k) // (math.factorial(i) * math.factorial(k - i))
        row.append(x)
    print(row)


Pascal_triangle(100)
Pascal_triangle(200)
```

本题运用了“math.factorial”计算阶乘，结合组合数公式“$C(k,i) = k!/(i!\,(k-i)!)$”生成帕斯卡三角的第 k 行。通过“for”循环（“range(k + 1)”）依次计算各项，使用列表（“row = []”）和添加操作（“row.append(x)”）保存结果，最后通过“print(row)”输出杨辉三角对应的一行，并调用函数分别计算“Pascal_triangle(100)”和“Pascal_triangle(200)”。

作业 4

4. Add or double

[20 points] If you start with `1` RMB and, with each move, you can either double your money or add another `1` RMB, what is the smallest number of moves you have to make to get to exactly *x* RMB? Here *x* is an integer randomly selected from `1` to `100`. Write a function `Least_moves` to print your results. For example, `Least_moves(2)` should print `1`, and `Least_moves(5)` should print `3`.

```
def Least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = x - 1
        moves += 1
    print(moves)
```

本题通过函数定义（“def Least_moves(x)”）、 “while”循环和条件判断（“if-else”），从目标金额 X 逆向计算到 1；利用取余运算符（“% 2”）判断数字的奇偶性，使用整数除法（“// 2”）和减法（“x - 1”）模拟逆向操作，并通过计数器（“moves += 1”）统计操作次数，最终用“print(moves)”输出得到目标金额所需的最少操作次数。

作业 5

5. Dynamic programming

Insert `+` or `-` operation anywhere between the digits `123456789` in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function `Find_expression`, which should be able to print every possible solution that makes the expression evaluate to a random integer from `1` to `100`. For example, `Find_expression(50)` should print lines include:

$$1 - 2 + 34 + 5 + 6 + 7 + 8 - 9 = 50$$

and

$$1 + 2 + 34 - 56 + 78 - 9 = 50$$

5.2 [5 points] Count the total number of suitable solutions for any integer *i* from `1` to `100`, assign the count to a list called `Total_solutions`. Plot the list `Total_solutions`, so which number(s) yields the maximum and minimum of `Total_solutions`?

```
import matplotlib.pyplot as plt
#生成所有可能的字符串
def generate_all():
    digits = "123456789"
    expressions = ["1"]
    for digit in digits[1:]:
        new_expressions = []
        for expr in expressions:
            new_expressions.append(expr + "+" + digit)
            new_expressions.append(expr + "-" + digit)
            new_expressions.append(expr + digit)
        expressions = new_expressions
#将所有可能的字符串转换成表达式并求值，筛选出符合范围的
    results = []
    for expr in expressions:
        value = eval(expr)
        if isinstance(value, int) and 1 <= value <= 100:
            results.append((expr, value))
    return results


def Find_expression(i):
    all_results = generate_all()
    found = False
    for expr, value in all_results:
        if value == i:
            print(f"{expr}={i}")
            found = True
    if not found:
        print(f"No solution found for {i}")
```

```
# 生成所有结果
all_results = generate_all()

# 统计 1~100 每个整数的解的数量
Total_solutions = []
for target in range(1, 101):
    count = 0
    for expr, value in all_results:
        if value == target:
            count += 1
    Total_solutions.append(count)

# 找出最大值和最小值对应的数字
max_count = max(Total_solutions)
min_count = min(Total_solutions)
max_numbers = [i+1 for i, c in enumerate(Total_solutions) if c == max_count]
min_numbers = [i+1 for i, c in enumerate(Total_solutions) if c == min_count]

print("Total_solutions =", Total_solutions)
print("最大解数量:", max_count, "对应数字:", max_numbers)
print("最小解数量:", min_count, "对应数字:", min_numbers)

# 绘图
x = list(range(1, 101))
plt.plot(x, Total_solutions, marker=".")
plt.xlabel("Target integer")
plt.ylabel("Number of solutions")
plt.title("Number of solutions for integers 1 to 100")
plt.grid(True)
plt.tight_layout()
plt.show()
```

本题通过字符串拼接（“expr + "+" + digit”）、“for”循环和列表（“append()”），实现数字之间添加加号、减号或直接连接的数学组合，利用穷举法生成所有可能的表达式；通过表达式求值（“eval(expr)”）计算每个表达式的结果，再使用“if”条件判断筛选出符合目标值的表达式；通过嵌套“for”循环和计数器（“count += 1”）统计 1～100 每个整数对应的解的数量；通过最大值、最小值函数（“max()、min()”）找出解数量最多和最少的目标整数；最后使用“matplotlib”绘图，将数学统计结果可视化。